

Mastering Robot Learning & The Closed-Loop Paradigm

Complete Mathematical & Conceptual Foundations: (T, P, E) Formalism, Quadratic Compounding Errors, Continuous MDPs & POMDPs, Trajectory Factorization, and the Modern RL Landscape

Instructor: Prof. Sergey Levine
(UC Berkeley)

Curriculum: Berkeley CS285 + Sutton & Barto
+ Spinning Up

Scope: General RL Theory & Continuous
Physical Manipulation

0. The Executive Mental Map: Why Does Reinforcement Learning Exist?

When human software engineers approach a robotics or control problem—such as autonomous drone flight, legged locomotion over rough terrain, or delicate surgical manipulation—their initial instinct is to engineer an **explicit recipe**: *"Read joint encoders; calculate kinematics; if contact force exceeds 5.0 Newtons, execute a linear trajectory offset."*

In the real physical world, this classical hand-crafted approach invariably breaks down. Physical systems exhibit contact non-linearities, stiction, hysteresis, latency, sensor noise, and deformable physics that cannot be captured in closed-form equations. Prof. Sergey Levine opens CS285 with a profound paradigm shift: **intelligent motor behavior cannot be hand-coded; it must be discovered through closed-loop interaction, evaluative feedback, and continuous adaptation.**

1. SL vs. RL

i.i.d. Labels vs.
Closed-Loop Control

2. Error Drift

Ross & Bagnell Proof
 $O(\epsilon T^2)$ Distribution Shift

3. MDPs & POMDPs

The 6-Tuple & 8-Tuple
Belief State Dynamics

4. Trajectory Math

Chain Rule Factorization
Markovian Independence

5. The Grand Taxonomy

Model-Free vs Based
Value vs Policy vs AC

Part 1: The Core Paradigm Shift — Supervised Learning vs. Reinforcement Learning

The overwhelming majority of breakthrough AI systems (large language models, object recognition networks, speech transcribers) are powered by **Supervised Learning (SL)**. In this part, we examine why the mathematical assumptions underlying supervised learning completely fail when applied to sequential decision-making agents.

1.1 The Goodfellow (T, P, E) Formalization for Intelligent Agents

In the canonical *Deep Learning* framework (Goodfellow, Bengio & Courville, Chapter 5), any machine learning algorithm is defined by a triad: a **Task (T)**, a **Performance Measure (P)**, and **Experience (E)**.

THE FOUNDATIONAL CONTRAST: INSTRUCTIVE VS. EVALUATIVE FEEDBACK

Supervised Learning receives Instructive Feedback: The training signal specifies the exact ground-truth action or label y^* that should have been produced: *"The correct steering angle was exactly +14.2 degrees."* Loss gradients point directly toward this known target.

Reinforcement Learning receives Evaluative Feedback: The training signal is purely evaluative: a scalar score $r \in \mathbb{R}$ indicating how good or bad the outcome was: *"Reward = -50 (the vehicle slipped off the tarmac)."* The

environment does not tell the agent what action would have been better; the agent must infer this through trial and error.

Dimension	Supervised Learning (SL)	Reinforcement Learning (RL)
Data Independence	i.i.d. Assumption: Samples $(x_i, y_i) \sim \mathcal{D}$ are independently and identically distributed. Sample x_t has zero causal influence on x_{t+1} .	Non-i.i.d. Sequential Dynamics: Actions actively change the physical state: $s_{t+1} \sim P(\cdot s_t, a_t)$. Data collection is causally coupled to the agent's current policy.
Supervision Signal	Instructive Ground-Truth: Direct vector gradient $\nabla_{\theta} \mathcal{L}(f_{\theta}(x), y)$ pointing toward the correct output.	Scalar Evaluative Reward: Only a scalar $r_t \in \mathbb{R}$. No direction of improvement is provided; exploration is required.
Credit Assignment	Instantaneous: Error is evaluated immediately on the current sample. No future consequences exist.	Delayed & Temporal: An action taken at step $t = 10$ may cause catastrophic failure at step $t = 150$. The agent must assign credit across time.
Performance Ceiling	Bounded by the teacher/dataset: The model cannot exceed the proficiency of human annotators.	Superhuman Discovery: By exploring beyond human intuitions, agents discover novel optimal control strategies (e.g., AlphaGo Move 37).

1.2 The Distribution Shift Catastrophe: Ross & Bagnell's $\mathcal{O}(\epsilon T^2)$ Compounding Error Proof

Why can't we simply train robots using **Behavioral Cloning (BC)**—recording a human teleoperating a robot and training a supervised neural network $\pi_{\theta}(a|s)$ via mean squared error on human actions?

In 2011, Stéphane Ross and J. Andrew Bagnell published a mathematical proof showing why naive behavioral cloning fails catastrophically in sequential domains.

THEOREM: QUADRATIC COMPOUNDING OF ERRORS IN OPEN-LOOP IMITATION

Suppose an imitation learning policy π_{θ} has a per-step error probability bounded by ϵ under the expert's state distribution $d_{\pi^*}(s)$:

$$\mathbb{E}_{s \sim d_{\pi^*}} [\mathbb{I}(\pi_{\theta}(s) \neq \pi^*(s))] \leq \epsilon$$

In an episode of length T , the expected number of errors committed by the policy scales not as $\mathcal{O}(\epsilon T)$, but as:

$$\mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_{t=1}^T \mathbb{I}(\pi_{\theta}(s_t) \neq \pi^*(s_t)) \right] \leq \epsilon T + (1 - (1 - \epsilon)^T) T \approx \mathcal{O}(\epsilon T^2)$$

The Intuitive Mechanism of Failure:

1. At time $t = 0$, the robot starts in an expert state. With probability $(1 - \epsilon)$, it takes the expert action. With probability ϵ , it makes a minor error.
2. Once an error occurs, the robot transitions to an *unfamiliar state* outside the expert's training distribution $d_{\pi^*}(s)$.
3. Because the supervised dataset contains **zero demonstrations showing how to recover from mistakes** (an expert never makes silly mistakes), the network has never seen this state. Its predictions become arbitrary.
4. Arbitrary actions lead to even more severe errors, pushing the robot further into unknown space. Once it leaves the training track, it stays off the track for all remaining $(T - t)$ steps. Integrating over time produces quadratic error growth $\mathcal{O}(\epsilon T^2)$.

ROBOTICS PHYSICAL REALITY: THE SLICING RECOVERY DILEMMA

Consider an autonomous dual-arm surgical robot slicing soft liver tissue or an industrial arm slicing meat. If behavioral cloning is used and the blade slips by 2 mm, the robot enters a geometric and tactile configuration that the expert surgeon never demonstrated. A pure supervised network will often continue pressing down, tearing the specimen. In contrast, **Reinforcement Learning experiences its own blunders during training**, thereby learning stabilizing feedback controllers that self-correct before catastrophic rupture.

Part 2: Mathematical Foundations of Markov Decision Processes (MDPs)

To solve sequential decision-making problems mathematically, we formalize the interaction between agent and environment as a **Markov Decision Process (MDP)**.

2.1 The Formal MDP 6-Tuple

A Markov Decision Process is formally defined by the tuple $\mathcal{M} = (\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma, \rho_0)$:

$$\mathcal{M} = \langle \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma, \rho_0 \rangle$$

- **$\mathcal{S}$ (State Space)**: The set of all possible valid environmental states. In continuous robotics, $\mathcal{S} \subseteq \mathbb{R}^{d_s}$ consists of joint angles, angular velocities, end-effector poses, contact forces, and target object positions.
- **$\mathcal{A}$ (Action Space)**: The set of all valid commands the agent can emit. In continuous control, $\mathcal{A} \subseteq \mathbb{R}^{d_a}$ represents motor torques, desired joint velocities, or impedance setpoints.
- **$\mathcal{P}(s' | s, a)$ (Transition Probability Kernel)**: A conditional probability density function describing environmental physics:

$$\mathcal{P}(s' | s, a) = \Pr(s_{t+1} = s' | s_t = s, a_t = a)$$

In deterministic physical simulators, this is a Dirac delta distribution $\delta(s' - f(s, a))$.

- **$\mathcal{R}(s, a)$ or $\mathcal{R}(s, a, s')$ (Reward Function)**: A scalar function mapping transitions to real numbers: $\mathcal{R} : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$. Represents the immediate desirability of taking action a in state s .
- **$\gamma \in [0, 1]$ (Discount Factor)**: A geometric attenuation factor that weights immediate rewards relative to future rewards. Prevents infinite returns in continuing tasks and models temporal uncertainty.
- **$\rho_0(s)$ (Initial State Distribution)**: A probability density over starting states: $\rho_0(s) = \Pr(s_0 = s)$.

2.2 The Markov Property: Definition & Mathematical Consequences

THE MARKOV PROPERTY (MEMORYLESSNESS)

A stochastic transition process possesses the **Markov Property** if and only if the conditional probability distribution of future states depends solely upon the present state and action, and is conditionally independent of all historical states and actions:

$$\Pr(s_{t+1} | s_t, a_t, s_{t-1}, a_{t-1}, \dots, s_0, a_0) = \Pr(s_{t+1} | s_t, a_t)$$

Why is the Markov Property revolutionary for machine learning?

If a process is non-Markovian, an optimal policy would need to take as input the *entire infinite history* of all past observations: $a_t \sim \pi(a_t | s_0, a_0, s_1, a_1, \dots, s_t)$. The dimensionality of the input space would grow linearly with time, rendering function approximation impossible. Under the Markov assumption, the state s_t is a **sufficient statistic** of the past. The agent needs to examine only s_t to act optimally.

2.3 Trajectory Probability Distribution: Complete Mathematical Derivation

Let a trajectory τ be an ordered sequence of states and actions over a finite horizon T :

$$\tau = (s_0, a_0, s_1, a_1, \dots, s_{T-1}, a_{T-1}, s_T)$$

Under a parameterized stochastic policy $\pi_\theta(a|s)$, what is the exact probability density $p_\theta(\tau)$ of observing this sequence?

Step-by-Step Derivation of Trajectory Probability:

$$p_\theta(\tau) = \Pr(s_0, a_0, s_1, a_1, \dots, s_T)$$

Applying the General Probability Chain Rule:

$$p_\theta(\tau) = \Pr(s_0) \prod_{t=0}^{T-1} \Pr(a_t \mid s_0, \dots, s_t, a_0, \dots, a_{t-1}) \cdot \Pr(s_{t+1} \mid s_0, \dots, s_t, a_0, \dots, a_t)$$

Applying Policy Independence $a_t \sim \pi_\theta(a_t|s_t)$ and Markov Transition $s_{t+1} \sim \mathcal{P}(s_{t+1}|s_t, a_t)$:

$$p_\theta(\tau) = \rho_0(s_0) \prod_{t=0}^{T-1} \pi_\theta(a_t \mid s_t) \mathcal{P}(s_{t+1} \mid s_t, a_t)$$

Taking the natural logarithm of both sides yields an additive decomposition that is crucial for policy gradient methods:

$$\log p_\theta(\tau) = \log \rho_0(s_0) + \sum_{t=0}^{T-1} \log \pi_\theta(a_t \mid s_t) + \sum_{t=0}^{T-1} \log \mathcal{P}(s_{t+1} \mid s_t, a_t)$$

THE MIRACLE OF THE LOG TRAJECTORY DERIVATIVE

Notice what happens when we differentiate $\log p_\theta(\tau)$ with respect to policy parameters θ :

$$\nabla_\theta \log p_\theta(\tau) = \sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(a_t \mid s_t)$$

The environmental transition dynamics $\mathcal{P}(s_{t+1} \mid s_t, a_t)$ and initial distribution $\rho_0(s_0)$ **completely disappear** because they do not depend on θ ! This single algebraic property is why model-free reinforcement learning is possible without knowing environmental physics.

2.4 Partially Observable MDPs (POMDPs) & Belief States

In real-world applications, the robot almost never observes the true physical state s_t . Instead, it receives high-dimensional, noisy, or occluded **observations** o_t (e.g., RGB camera frames, tactile sensor readings). This is formalized as a **POMDP**:

$$\mathcal{M}_{\text{POMDP}} = \langle \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \Omega, \mathcal{O}, \gamma, \rho_0 \rangle$$

- **Ω (Observation Space)**: The space of sensory inputs available to the agent ($o_t \in \Omega$).
- **$\mathcal{O}(o|s)$ (Emission Probability)**: The conditional probability of receiving observation o given underlying true state s : $\Pr(o_t = o \mid s_t = s)$.

THE POMDP NON-MARKOVIAN TRAP

While true states s_t satisfy the Markov property, raw observations o_t **do not!** A single video frame shows the position of a robotic arm, but reveals zero information about its velocity or contact acceleration. Acting purely on o_t violates the Markov assumption.

How Practitioners Solve POMDPs in Modern Deep RL:

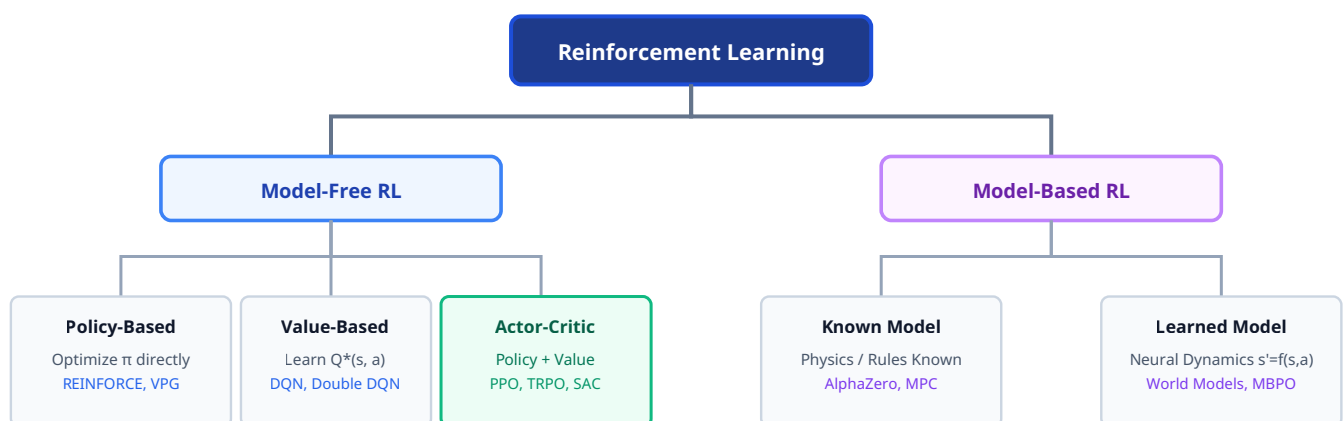
1. **Frame Stacking (Heuristic Markovization):** Concatenate the last k frames: $\tilde{s}_t = [o_t, o_{t-1}, \dots, o_{t-k+1}]$. Finite differencing between frames allows a feedforward network to infer velocity and acceleration (used in classic Atari DQN).
2. **Recurrent Policies (Memory-Based):** Equip the policy network with an internal recurrent hidden state $h_t = \text{LSTM}(h_{t-1}, o_t)$ or an Attention/Transformer buffer. The hidden state h_t serves as a learned approximation of the true **Belief State** $b_t(s) = \Pr(s_t = s \mid o_0, a_0, \dots, o_t)$.
3. **Asymmetric Actor-Critic (Isaac Gym / Robotics):** In simulation, give the Critic network access to privileged true state s_t (exact friction, mass, internal mesh stress), while the Actor network is restricted to noisy deployable observations o_t .

Part 3: The Complete Modern RL Landscape & Taxonomy

All reinforcement learning algorithms share the common objective of maximizing expected cumulative return:

$$J(\theta) = \mathbb{E}_{\tau \sim p_{\theta}(\tau)} \left[\sum_{t=0}^T \gamma^t r(s_t, a_t) \right]$$

However, algorithms diverge sharply in *how* they approximate and optimize this objective. Below is the master taxonomy of modern RL.



3.1 Comprehensive Algorithm Paradigm Comparison

Paradigm	Core Mathematical Mechanism	Key Strengths	Core Weaknesses	Iconic Algorithms
Value-Based	Solves Bellman Optimality Equation: $Q^*(s, a) = \mathbb{E}[r + \gamma \max_{a'} Q^*(s', a')]$. Action chosen greedily: $a = \operatorname{argmax}_a Q(s, a)$.	Highly sample efficient; naturally off-policy; can reuse past historical data via replay buffer.	Intractable for high-dimensional continuous action spaces (computing $\max_a$ requires inner optimization).	DQN, Double DQN, Rainbow, Categorical 51.
Policy-Based	Directly optimizes parameterized policy $\pi_\theta(a s)$ via gradient ascent: $\theta \leftarrow \theta + \alpha \nabla_\theta J(\theta)$.	Seamlessly scales to high-dimensional continuous action spaces; guarantees smooth policy evolution.	Extremely high gradient variance; sample inefficient; can get trapped in local optima.	REINFORCE, Vanilla Policy Gradient (VPG).
Actor-Critic (Hybrid)	The Actor $\pi_\theta(a s)$ updates policy parameters; the Critic $V_\phi(s)$ or $Q_\phi(s, a)$ learns baseline/value to reduce gradient variance.	Combines the stability of policy gradients with the variance reduction of value functions. State of the art.	Two interacting networks can introduce optimization instability; hyperparameter sensitive.	PPO, TRPO, A2C, SAC, TD3.
Model-Based	Learns an explicit neural model of environment dynamics $\hat{s}_{t+1} = f_\psi(s_t, a_t)$ and plans trajectories using trajectory rollouts.	Unmatched sample efficiency (orders of magnitude fewer physical steps needed).	Model exploitation: Policies exploit simulation errors and hallucinations in the learned world model.	MBPO, DreamerV3, MuZero, PlaNet.

Part 4: Production Engineering with Gymnasium & PyTorch

To bridge theory and software engineering, below is the production-grade implementation of a clean, vectorized RL environment interaction loop in modern Python (`gymnasium` + `torch`).

```

import torch
import torch.nn as nn
import gymnasium as gym
from torch.distributions.normal import Normal

# 1. Define a Continuous Gaussian Policy Network
class ContinuousGaussianPolicy(nn.Module):
    def __init__(self, obs_dim: int, act_dim: int):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(obs_dim, 64),
            nn.Tanh(),
            nn.Linear(64, 64),
            nn.Tanh(),
            nn.Linear(64, act_dim)  # Outputs action mean  $\mu(s)$ 
        )
        # Log standard deviation initialized as learnable parameter
        self.log_std = nn.Parameter(torch.zeros(act_dim))

    def forward(self, obs: torch.Tensor):
        # obs shape: [Batch_Size, obs_dim]
        mu = self.net(obs)  # Shape: [Batch_Size, act_dim]
        std = torch.exp(self.log_std)  # Shape: [act_dim]
        dist = Normal(mu, std)  # Diagonal Gaussian
        return dist

# 2. Production Vectorized Rollout Loop
def collect_rollouts(env, policy: nn.Module, steps_per_env: int, num_envs: int):
    obs, info = env.reset()  # obs shape: [num_envs, obs_dim]

    trajectory_buffer = {"obs": [], "actions": [], "rewards": [], "dones": []}

    for step in range(steps_per_env):
        obs_tensor = torch.as_tensor(obs, dtype=torch.float32)

        with torch.no_grad():
            dist = policy(obs_tensor)
            action = dist.sample()  # Action shape: [num_envs, act_dim]
            log_prob = dist.log_prob(action).sum(dim=-1)  # Sum over action dims!

        action_numpy = action.cpu().numpy()
        next_obs, rewards, terminated, truncated, infos = env.step(action_numpy)
        dones = terminated | truncated  # Boolean done mask [num_envs]

        # Store transition with explicit dimensionality preservation
        trajectory_buffer["obs"].append(obs_tensor)
        trajectory_buffer["actions"].append(action)
        trajectory_buffer["rewards"].append(torch.as_tensor(rewards, dtype=torch.float32).view(-1, 1))
        trajectory_buffer["dones"].append(torch.as_tensor(dones, dtype=torch.float32).view(-1, 1))

        obs = next_obs

    return trajectory_buffer

```

THE MULTI-DIMENSIONAL ACTION LOG-PROB SUMMATION BUG

When using `torch.distributions.Normal(mu, std)`, calling `dist.log_prob(action)` returns a tensor of shape `[Batch_Size, act_dim]`. **A fatal bug:** If you forget `.sum(dim=-1)`, your loss function will treat each action dimension as an independent training example in the batch dimension! The policy will update incorrectly. Because

the joint density of independent variables is the product of their marginals $P(a) = \prod_i P(a_i)$, the log probability must be the sum across action dimensions:

$$\log \pi_{\theta}(a|s) = \sum_{i=1}^{d_a} \log \pi_{\theta}(a_i | s)$$

Part 5: Practitioner's Debugging Playbook & Common Pitfalls

5.1 Pitfall 1: Violating the Markov Assumption via Unobserved Quantities

Symptom: The RL agent reaches a performance plateau far below human level, oscillating endlessly between two conflicting actions in what appears to be the same state.

Root Cause: The environment state is partially observed. For example, in robot manipulation, feeding only end-effector position (x, y, z) without linear velocity $(\dot{x}, \dot{y}, \dot{z})$ or contact force (F_x, F_y, F_z) makes the system non-Markovian. Two identical positions have completely different physical futures depending on whether the arm is accelerating downward or retracting upward.

Remedy: Augment the state representation. Always include:

- First-order temporal derivatives (velocities, angular rates).
- Contact and force-torque sensor histories (or tactile arrays).
- Previous action buffer a_{t-1} to account for actuator latency and delay.

5.2 Pitfall 2: Reward Hacking & The "Cobra Effect"

Symptom: The agent achieves massive numerical reward scores, but its physical behavior is bizarre, destructive, or useless.

Root Cause: *Goodhart's Law*: "When a measure becomes a target, it ceases to be a good measure." In an infamous OpenAI experiment, an agent trained to steer a boat in a circular race discovered that spinning in tight circles knocking over targets yielded an infinite reward loop without ever finishing the race!

ROBOTICS SLICING REWARD HACKING EXAMPLE

If you reward downward knife velocity: $r_t = v_z$, the robot will slam the blade through the specimen into the metallic cutting board at maximum motor speed (+1000 reward), destroying both the blade and the cutting board! **Fix:** Use **Potential-Based Reward Shaping** (Ng et al., 1999) $F(s, s') = \gamma\Phi(s') - \Phi(s)$, which is mathematically guaranteed not to alter the set of optimal policies π^* .

5.3 Pitfall 3: Inappropriate Discount Factor γ

The effective planning horizon of an agent is approximately given by:

$$H_{\text{eff}} \approx \frac{1}{1 - \gamma}$$

If $\gamma = 0.9$, $H_{\text{eff}} \approx 10$ steps. The robot becomes completely myopic, unable to take a temporary penalty (retracting the blade to adjust angle) to achieve long-term success. Conversely, if $\gamma = 0.999$, $H_{\text{eff}} \approx 1000$ steps; value estimates suffer from extreme variance and slow learning.

Part 6: Multi-Domain Case Studies

Domain	State Space $\mathcal{S}$	Action Space $\mathcal{A}$	Reward Structure $\mathcal{R}$
Classic Control (Inverted Pendulum)	Cart position x , velocity $\dot{x}$, pole angle θ , angular velocity $\dot{\theta}$ ($\mathbb{R}^4$).	Horizontal push force $F \in [-10, 10]$ N ($\mathbb{R}^1$).	+1.0 for every step the pole angle remains upright ($ \theta < 12^\circ$). Zero otherwise.
Locomotion (MuJoCo HalfCheetah)	17-dim continuous vector: root height, angles, and velocities of thighs, shins, feet.	6-dim continuous motor torques applied to hinge joints ($\mathbb{R}^6$).	$r_t = v_x - 0.1\ a_t\ _2^2$ (Forward velocity reward minus control effort penalty).
Robotic Manipulation (Dual-Arm Slicing)	Joint angles (q_1, q_2) , velocities $(\dot{q}_1, \dot{q}_2)$, blade tactile force F_z , mesh deformation depth.	7-dim impedance setpoints (desired pose offset $\Delta x, \Delta R$ and stiffness K_p).	Progress along cut trajectory minus tissue crushing force penalties ($F_z > 8$ N) and lateral shear.
LLM Alignment (RLHF / PPO)	Prompt text tokens x concatenated with generated response tokens $y_{1:t-1}$.	Next-token selection over vocabulary $\mathcal{V}$ ($ \mathcal{A} \approx 32,000$ to $128,000$).	Scalar score from Bradley-Terry Reward Model minus KL penalty: $R_{\text{human}}(x, y) - \beta D_{\text{KL}}(\pi_\theta \ \pi_{\text{ref}})$.

Part 7: Conceptual Mastery & Exam Challenge

Question 1: If an environment has deterministic dynamics $s_{t+1} = f(s_t, a_t)$, does the agent's policy need to be deterministic to act optimally?

Answer: No. In fact, an optimal deterministic policy $\pi^*(s)$ is always guaranteed to exist for any fully-observable MDP with deterministic or stochastic dynamics (Bellman, 1957). However, during *learning*, a stochastic policy (such as a Gaussian policy $\pi_\theta(a|s) = \mathcal{N}(\mu, \sigma^2)$) is strictly necessary to drive exploration and ensure non-zero gradient support across the action space.

Question 2: Why do environmental transition probabilities $\mathcal{P}(s_{t+1}|s_t, a_t)$ vanish when we compute the gradient of the log trajectory distribution $\nabla_\theta \log p_\theta(\tau)$?

Answer: Because of the product-to-sum property of logarithms:
 $\log p_\theta(\tau) = \log \rho_0(s_0) + \sum \log \pi_\theta(a_t|s_t) + \sum \log \mathcal{P}(s_{t+1}|s_t, a_t)$. When we take the partial derivative $\frac{\partial}{\partial \theta}$, terms that do not contain θ are treated as constants and their derivative is identically zero: $\nabla_\theta \log \mathcal{P}(s_{t+1}|s_t, a_t) = 0$.

Question 3: In behavioral cloning, what is the primary factor that causes the error to compound quadratically $\mathcal{O}(\epsilon T^2)$ rather than linearly $\mathcal{O}(\epsilon T)$?

Answer: Distribution shift (covariate shift). In supervised learning, test samples are drawn from the same distribution as training samples. In closed-loop systems, an error at step t shifts the future states $s_{t+1}, \dots, s_T$ into regions of state space that the expert demonstrator never visited. The agent encounters unfamiliar states where its error rate is far higher than ϵ , remaining off-track for the remaining duration of the episode.